# SI-Mapper: Agentic AI for Automated HVAC Semantic Modeling

From BACnet Data and Engineering Drawings to ASHRAE 223P Ontologies

Juan [Last Name]
*[Affiliation]*

March 2026

**Abstract**

This report presents SI-Mapper, an agentic AI system that automates the creation of ASHRAE 223P semantic building models from HVAC engineering drawings and BACnet point data. We describe the problem of manual BACnet point mapping — a costly, labor-intensive process that blocks building software interoperability and grid integration — and present an AI-driven solution that replaces weeks of expert engineering work. The report traces the development through nine experimental iterations, from simple function-calling approaches to the final skills-based single master agent architecture, and provides a framework for comparing AI-generated results against human engineer baselines.

## Contents

# 1 Introduction and Context

This section establishes the conceptual foundation for understanding SI-Mapper. It explains what building ontologies are, why a standardized vocabulary for building systems matters, and why the ASHRAE 223P standard was chosen as the target output format. Readers do not need a technical background to follow this discussion.

## 1.1 The Semantic Metadata Gap in Buildings

Every commercial building contains hundreds of sensors, actuators, and control points. A modern office block may have thousands of temperature sensors, pressure transducers, damper position indicators, and setpoint registers — each one a source of valuable operational data. Yet despite this wealth of instrumentation, the data is largely inaccessible to software that was not purpose-built for the specific building it operates in.

The reason is a fundamental naming problem. Each Building Management System (BMS) vendor uses its own proprietary conventions for labelling control points. A temperature sensor named `2500.AI11` in one system becomes `RTU-3/SA-T` in another. Neither label carries enough information for software to determine automatically what physical equipment the point belongs to, what role the measurement plays, or how that equipment connects to other components. Without that context, building software cannot be deployed without a building-specific customization step.

As the United States Department of Energy states directly, "building software is not 'plug and play' — technicians must manually map sensors and actuators to software inputs through tedious point mapping processes that vary building-by-building." This manual, building-by-building effort is the central barrier this report addresses.

The solution is a *standardized metadata layer* — a machine-readable description of what every sensor and actuator means, which equipment it belongs to, and how that equipment connects to the rest of the system. This is what building ontologies provide.

## 1.2 Building Ontologies: A Standardized Language for Buildings

A building ontology is a shared, machine-readable vocabulary that describes building systems, their components, and how they connect. Think of it as a dictionary that any software application can consult: if a sensor is declared to be a supply air temperature measurement on the outlet of a fan coil unit, any compliant application can understand that without manual configuration.

Four major building ontologies have emerged over the past decade. Each reflects different design priorities and is backed by a different community.

**Table 1:** The four major building ontology standards.

| Ontology | Governed By | Approach | Focus |
|---|---|---|---|
| Project Haystack | Haystack Connect | Tag-based dictionary | Equipment types, sensor tags |
| Brick Schema | Open consortium | RDF/OWL semantic graph | Equipment, sensors, spatial hierarchy |
| RealEstateCore | RealEstateCore Consortium | OWL ontology | Property management + technical sys |
| ASHRAE 223P | ASHRAE + DOE/NREL | Formal OWL ontology | Topology, connections, telemetry |

**Project Haystack** is the oldest and most widely deployed approach. It uses a tag-based system where sensors are described by a set of keywords (e.g., `air temp sensor`). Its informality makes it easy to adopt but difficult to enforce consistently across projects.

**Brick Schema** builds formal semantic rules on top of a Haystack-compatible vocabulary. It uses the RDF/OWL knowledge representation standard, making it more consistent and machine-interpretable than Haystack. Brick is production-ready and in active deployment across research institutions and building operators.

**RealEstateCore** focuses on the property management perspective — occupancy, asset management, and energy performance — with significant overlap into technical building systems. It is used mainly in the European commercial real estate sector.

**ASHRAE 223P** is the most rigorous of the four. It defines not just equipment types but full system topology: how equipment connects, what flows between components, and how those connections link to live sensor data. ASHRAE's BACnet committee, Project Haystack, and Brick Schema are actively collaborating to integrate their vocabularies into ASHRAE 223P as the unifying standard.

A 2025 systematic comparison of these four ontologies found that no single standard covers all needs in practice. A complete building model is expected to be "a mix of ASHRAE 223P, Brick, QUDT, RealEstateCore, and possibly additional standards." ASHRAE 223P is designed as the semantic backbone that ties the others together.

### 1.3 ASHRAE Standard 223P

ASHRAE Standard 223P is a proposed ASHRAE standard that formally defines a knowledge representation framework for building systems. Its goal is to enable software applications to determine the meaning and context of building data without manual, building-specific configuration.

**Current status:** The standard is in its second advisory public review as of 2025–2026. It is not yet ratified. The project receives $1.3 million in Department of Energy funding and is led by the National Renewable Energy Laboratory (NREL), with participation from Lawrence Berkeley National Lab, Pacific Northwest National Lab, NIST, Colorado School of Mines, and Cornell University.

ASHRAE 223P organizes building system information around six core concepts, described in Table 2.

**Table 2:** The six core concepts of ASHRAE 223P.

| Concept | Definition | Example |
|---|---|---|
| Type | Well-defined classes with formal rules governing usage | Fan, Pump, CoolingCoil |
| Topology | How equipment and spaces connect | Fan outlet → Duct → Coil inlet |
| Composition | Hierarchical grouping of entities | AHU contains Fan, Coil, Damper |
| Telemetry | Observable properties with external data references | Supply air temp → BACnet object 2500.AI11 |
| Characteristics | Descriptive attributes with units | Rated airflow in $m^3/s$ |
| Medium | Substance conveyed through connections | Chilled water, Supply air, Electricity |

**Topology** is the most distinctive concept in ASHRAE 223P. Where other ontologies describe what equipment exists, 223P also describes how equipment connects. Every connection has a direction (inlet, outlet, or bidirectional), a physical path (duct, pipe, or wire), and a medium (air, water, electricity). This means an ASHRAE 223P model captures not just "there is a fan and a coil" but "air flows from the fan outlet through a duct segment into the coil inlet, conveying Supply Air at design conditions."

**Telemetry and external references** allow live BACnet sensor data to be explicitly linked to semantic properties in the model. A supply air temperature sensor in the model can declare that its value comes from `BACnet object 2500.AI11` — closing the loop between the abstract ontology and the live building management system.

**Why SI-Mapper chose ASHRAE 223P:** It is the only standard that simultaneously captures full HVAC topology, links BACnet points to semantic equipment properties, and is explicitly designed for advanced building controls and grid services. Its backing by DOE,

NREL, and ASHRAE's standardization process makes it the most credible long-term choice for interoperability.

### 1.4 Advantages and Barriers of Semantic Building Models

Creating a semantic building model is not a trivial investment. Understanding why organizations would make that investment — and why they currently do not — frames the business case for automation tools like SI-Mapper.

**Advantages of semantic building models:**

- **Software portability:** applications built on a standard ontology can work across buildings without building-specific customization, just as web browsers work across websites.
- **Advanced analytics:** fault detection, automated commissioning, and predictive maintenance all require knowing not just sensor values but what those values mean and how equipment relates to each other.
- **Grid services enablement:** Virtual Power Plant (VPP) and Non-Wire Alternative (NWA) programs require machine-readable building load data; semantic models provide this in a standardized form.
- **Reduced integration costs:** once a semantic model exists, any compliant application can consume it without a custom integration project.
- **Digital twin support:** the semantic layer is the connective tissue between physical building systems and their digital representations.

**Barriers to adoption:**

- **High creation cost:** building a semantic model today requires specialized domain expertise and significant labor — often weeks of work by a qualified HVAC engineer.
- **No automated tooling for existing buildings:** most available approaches are manual or semi-manual, requiring expert knowledge of both HVAC systems and ontology standards.
- **Fragmented standards landscape:** organizations must choose between partially overlapping standards with no single dominant solution.
- **Model maintenance:** models drift from reality as building systems are modified, requiring ongoing expert effort to keep them current.
- **Low industry adoption:** despite clear benefits, the broader industry has been slow to adopt formal ontologies, creating a chicken-and-egg problem for software vendors.

The high cost of *creating* semantic models — not deploying them — is the central barrier. A semantic model that would take a qualified engineer several weeks to create manually could enable years of automated analytics and grid integration. The investment is worthwhile; the bottleneck is the creation step. This is the problem SI-Mapper is designed to solve.

## 2 Description of the Problem

Section 1 established that building semantic models unlock significant value but are costly to create. This section examines the cost problem in concrete terms. We describe what BACnet points are, quantify the labor required to map them, identify the research gap that has prevented an automated solution, and explain why the same problem blocks the emerging Virtual Power Plant sector.

## 2.1 The BACnet Point Mapping Problem

A Building Automation System (BAS) exposes operational data through a hierarchy of *points* — individual sensors, actuators, setpoints, alarms, and status flags. In BACnet (the dominant open protocol for commercial building controls), each point is a typed object — an Analog Input, Binary Output, Analog Value, and so on — with a unique address on the network. A typical commercial office building has between 500 and 5,000 BACnet points. A large hospital or campus facility may have tens of thousands.

The *mapping problem* is this: a raw list of BACnet points contains no standard information about what physical equipment each point belongs to, what role it plays in the system, or how that equipment connects to other components. A point named `AHU-1.SA-T` tells a human engineer "probably supply air temperature on Air Handling Unit 1," but only if the engineer already knows the naming convention used by that particular BMS installer. Conventions differ between contractors, between projects, and even between systems within the same building.

Reconstructing the full picture — which points belong to which equipment, what each point means, and how the equipment connects into a complete HVAC system — requires reading as-built HVAC drawings, interrogating BMS programming, and applying expert engineering judgment. This process is manual, time-consuming, and dependent on domain expertise that is increasingly scarce.

Modbus, an even older and simpler protocol used in legacy HVAC controllers, meters, and industrial equipment, exposes data as bare numbered registers with no descriptive metadata at all — making the mapping challenge even more difficult than BACnet.

## 2.2 The Cost of Manual Mapping

Manual BACnet point mapping and BAS commissioning are widely documented as major cost drivers in building construction and retrofit projects. Table 3 summarizes the key figures from industry research and published case studies.

To illustrate the compounding nature of these costs: the 150,000 sq ft school in the case study paid $225,000 for commissioning up front, then incurred approximately $175,000 in unplanned first-year expenses due to inadequate point documentation — nearly doubling the effective cost of the commissioning phase. According to NREL (2021), complete BAS installation averages $7.76 per square foot across the United States, with the legacy systems integration premium adding a further 20 to 40 percent on retrofit projects.

Manual approaches are also incomplete by design. Industry practice typically tests only 10 to 20 percent of terminal units during commissioning; every 10 percent reduction in testing coverage correlates to approximately a 6 to 8 percent increase in post-occupancy issues. The result is a pervasive, chronic problem: buildings begin their operational life with under-documented, partially verified control systems, and the cost of correction grows over time.

Industry practitioners report per-point costs of $50–$200 for full commissioning including point mapping, though exact figures vary by project type, contractor, and system complexity. A DOE and ACEEE 2024 paper describes manual point mapping as "labor intensive and costly, presenting a major impediment to innovations in building control" — validating the scale of the problem even where precise figures are contractor-dependent.

## 2.3 The Research Gap

Despite the scale and economic impact of the manual mapping problem, no widely-deployed automated solution exists.

A systematic review published in *Automation in Construction* (ScienceDirect, S0926580517300018) found that the field lacks:

- a complete problem formulation with integrated solution approaches,

**Table 3:** Key cost and effort data for manual BAS commissioning and point mapping.

| Cost Item | Figures | Source |
|---|---|---|
| BAS commissioning as % of construction | 0.5%–1.5% of total; 8%–15% of BAS project value | Building Commissioning Association |
| Case study: 150,000 sq ft school | $225,000 commissioning ($1.50/sq ft) | pingcx.com |
| Complete BAS installation average | $7.76/sq ft (range: $5.20–$14.18) | NREL (2021) |
| Legacy BMS integration premium | +20%–40% over base cost | envigilance.com (2023) |
| Additional service calls from poor mapping | 12–15 extra calls/year; +30%–45% troubleshooting time | pingcx.com |
| Energy performance drift from poor commissioning | 15%–30% energy waste in first 3 years; $0.20–$0.50/sq ft/year | pingcx.com |
| First-year unplanned expenses (case study school) | $175,000 — nearly equal to original commissioning cost | pingcx.com |
| HVAC technician shortage (2024–2025) | 110,000 unfilled positions | leads4build.com |

- standardized test datasets and performance metrics, and
- methods well-aligned with real-world mapping requirements.

The DOE and ACEEE 2024 conference paper on digital and interoperable buildings independently reaches the same conclusion: manual point mapping is "a major impediment to innovations in building control," and no automated solution has achieved production-level adoption.

The Building Commissioning Association's 2024 definitions confirm that point-to-point checking — the manual verification of every sensor and actuator in a BAS — remains a required step on every new BAS project. It is not an edge case or a legacy practice; it is standard procedure.

This gap — a documented, expensive, universally-acknowledged problem with no automated solution — validates SI-Mapper's research contribution. The closest academic work (BrickLLM, 2025) addresses natural language description to Brick ontology conversion but requires accurate text input and does not address drawing recognition, BACnet CSV parsing, or ASHRAE 223P topology generation.

## 2.4 Why This Matters for the Energy Grid

The point mapping problem is not only a building operations challenge. It is a direct barrier to the energy transition.

**Non-Wire Alternatives (NWA)** and **Virtual Power Plants (VPP)** aggregate distributed energy resources — including commercial building HVAC loads — to provide grid services equivalent to building new power plants or transmission lines. HVAC systems consume approximately 40 percent of total commercial building energy and can be modulated to respond to grid signals, shifting load away from peak demand periods. In aggregate, this flexibility represents enormous potential grid capacity.

The VPP market is growing rapidly. North American VPP programs totaled 37.5 GW of flexible behind-the-meter capacity as of 2025, up 14 percent from 2024, with active deployments rising more than 33 percent in the same period. Policy support is accelerating: ten state legislatures introduced VPP bills in 2024; Massachusetts utilities are implementing a $50 million Grid Services Compensation Fund for non-wire alternatives; and the California Energy Commission awarded a $2 million EPRI grant specifically to demonstrate VPPs for commercial buildings including HVAC controls.

Despite this growth, the DOE VPP Liftoff 2025 report identifies a critical structural barrier: "No industry standardization exists. VPP vendors have been building their solutions from the ground up, then working on a case-by-case basis with utilities to develop customized solutions, resulting in rigid systems that can't easily be replicated in new areas."

The barrier is technical and it is the same one described throughout this section. The chain of causality is straightforward:

1. Building HVAC system data is not machine-readable in a standard form.
2. Each building requires custom integration work to connect to a VPP platform.
3. Custom integration costs $5,000–$15,000 per building, or more for complex facilities.
4. High per-building cost prevents VPP aggregators from enrolling the thousands of buildings needed for reliable grid capacity.
5. Without scale, VPP programs cannot deliver the consistent grid services that utilities require.

If buildings had ASHRAE 223P semantic models, VPP platforms could automatically discover what controllable loads exist, query their current state, and dispatch them — all without a custom integration project. The semantic model becomes the interoperability layer that eliminates step 2 from the chain above.

This positions SI-Mapper as more than a productivity tool for HVAC engineers. It is potentially a piece of critical infrastructure for the energy transition: the automation layer that creates the semantic data required to connect commercial building flexibility to the grid at scale.

# 3 A Solution: Agentic AI for Building Semantic Modeling

The previous section established that manual BACnet point mapping is costly, labor-intensive, and resistant to automation with existing tools. This section explains why agentic AI is the right approach to solving it, situates SI-Mapper within the landscape of prior work, and introduces the high-level pipeline that the implementation chapters will describe in detail.

## 3.1 Existing Approaches and Their Limitations

Several approaches have been applied to the point mapping and building digitization problem. Each addresses part of the challenge but leaves a significant gap. Table 4 summarizes the state of the art.

The gap is clear: no prior system combines visual HVAC drawing analysis, automated BACnet CSV point-to-equipment mapping, full ASHRAE 223P topology generation, and iterative self-correction in a single pipeline. The closest academic work — BrickLLM (2025, ScienceDirect S2352711025000883) — addresses only natural language input to Brick RDF conversion and does not address the drawing recognition problem, BACnet CSV parsing, or ASHRAE 223P topology generation.

**Table 4:** Existing approaches to building point mapping and semantic model generation.

| Approach | What It Does | Why It Falls Short |
|---|---|---|
| Manual expert mapping | Engineer maps all points by hand using drawings and BMS documentation | $50–$200 per point; weeks per building; requires scarce HVAC expertise; error-prone |
| Vendor proprietary tools | BMS vendors (e.g., Siemens, Johnson Controls) provide semi-automated configuration within their ecosystem | Vendor lock-in; does not produce semantic models; tied to new installations only; no interoperability |
| Tag-based normalization | Rule-based scripts parse point names using regex or keyword matching | Brittle; breaks with naming convention variations; captures equipment types only, no topology |
| Building Information Modeling (BIM) | Architectural models capture equipment geometry and spatial layout | Created during design; diverges from as-built reality; no BACnet operational data; requires BIM expertise |
| BrickLLM (2025) | Large language model converts natural language building descriptions to Brick RDF | Requires accurate natural language input; no visual drawing analysis; no BACnet integration; produces Brick, not ASHRAE 223P |
| LLM point classification | LLMs classify individual BACnet point names into ontology tags | Classifies individual points only; does not reconstruct topology; not end-to-end |

### 3.2 What is Agentic AI?

To understand why SI-Mapper represents a genuine advance, it helps to understand what distinguishes agentic AI from earlier approaches.

Traditional AI systems answer single, isolated questions: classify this image, summarize this document, answer this query. *Agentic AI* systems go further: they can autonomously plan, execute, and verify multi-step tasks without human intervention at each step. They use external tools, maintain state across multiple turns, and can loop — retrying, correcting errors, and adapting — until a goal is achieved. The distinction matters because building semantic model generation is not a single-step question; it is a multi-stage workflow that requires reasoning about drawings, structured data, domain rules, and the validity of a generated output.

The industry recognizes three generations of AI deployment: **V1** (function calling) treated AI as a smart query interface with human-coded business logic. **V2** (workflows) embedded AI into predefined sequences, with humans designing the flow. **V3** (agentic) allows AI to autonomously plan the steps, execute them, and evaluate the results — with humans setting goals and reviewing outcomes rather than scripting every action. SI-Mapper operates at the V3 level.

Industry adoption of agentic architectures is accelerating. Gartner reported a 1,445% surge in multi-agent system inquiries between Q1 2024 and Q2 2025. The Model Context Protocol (MCP), developed by Anthropic and now supported by OpenAI, Microsoft, and Replit, has emerged as the de-facto standard for connecting AI agents to external tools and data sources — providing the interoperability layer that makes agentic systems practical across heterogeneous environments.

### 3.3 SI-Mapper: An Agentic Approach

SI-Mapper takes two raw building data inputs — HVAC engineering drawings (PNG or PDF) and BACnet point exports (CSV files) — and produces two outputs: an ASHRAE 223P semantic model in Turtle (TTL) format, and an interactive graph visualization of that model in a web interface.

The entire pipeline is orchestrated by a single AI agent guided by domain-specific skill instructions. Figure 1 illustrates the data flow.

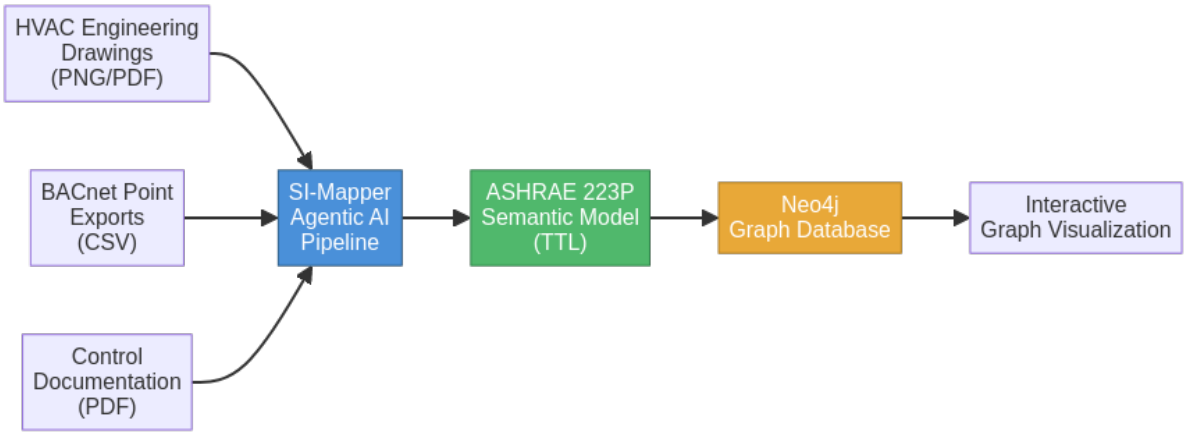


**Figure 1:** SI-Mapper data pipeline — from raw building data to semantic model and graph visualization.

The key enabler is multimodal AI. Recent large language models — including Gemini 2.5 and Claude Sonnet — can analyze engineering drawings directly, identifying equipment symbols, ductwork connections, and spatial relationships from visual inputs. HVAC schematic diagrams, with their regular symbol vocabulary and consistent layout conventions, are more tractable for AI recognition than general architectural floor plans. The insight from SI-Mapper's development is that standard models underperform on domain-specific drawing recognition until guided by detailed, domain-specific instructions — the *skills* architecture described in Section 4.

SI-Mapper's novelty lies in combining four capabilities that no prior system integrates:

1. **Multimodal drawing recognition:** AI analysis of HVAC engineering drawings to extract equipment and ductwork topology.
2. **BACnet CSV mapping:** automated assignment of BACnet point data to the equipment identified in the drawings.
3. **ASHRAE 223P generation:** production of a formally valid ASHRAE 223P ontology including ConnectionPoints, Connections, mediums, and telemetry links.
4. **Iterative self-correction:** the agent compares its generated model against validation rules, identifies errors, and corrects them autonomously — looping until the model is valid.

This approach replaces weeks of expert engineering effort with an automated pipeline that runs in minutes, producing a machine-readable semantic model that unlocks software portability, advanced analytics, and grid integration for the building. Section 4 describes the engineering decisions and experimental iterations that led to this architecture.

## 4 Implementation

### 4.1 Development Methodology: Nine Experimental Iterations

The development of SI-Mapper followed an iterative experimental methodology. Over nine distinct iterations, the system architecture evolved from simple function-calling scripts to a

sophisticated skills-based agentic AI system. Each iteration was driven by the failures and limitations of the previous one.

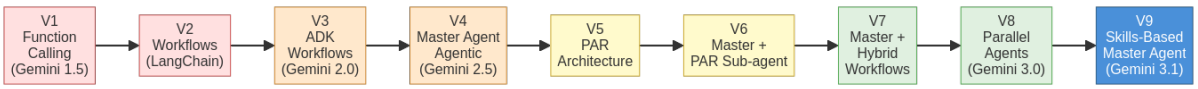


**Figure 2:** Evolution of SI-Mapper architecture across nine experimental iterations.

**Iteration 1 — LLMs as Function Providers (Gemini 1.5 Flash).** The first approach used LLMs as providers of intelligence embedded inside conventional Python functions: the model was called for specific tasks such as image recognition or text generation, while all business logic remained hardcoded by the developer. This approach was straightforward but fundamentally limited — the LLM had no awareness of the application context and could not make autonomous decisions. Every new capability required writing new code around it. LangChain was used as the integration library at this stage.

**Iteration 2 — Workflow-Based Agents (Gemini 1.5 Flash).** The second iteration introduced workflows to define agent behavior as a sequence of ordered steps, making it possible to better exploit the LLM's capabilities. However, the business logic was still hardcoded inside the workflow definitions, and the agent remained unable to adapt to unexpected situations. LangGraph was evaluated at this stage but was abandoned due to its high commercial licensing costs, which made it unsuitable for production deployment.

**Iteration 3 — ADK Workflows (Gemini 2.0 Flash).** The third iteration migrated to Google's Agent Development Kit (ADK) while retaining the workflow-based execution model. A local RAG system was introduced to provide agents with on-demand access to documentation for HVAC equipment classification. ADK had been very recently released at this point, with limited documentation and community examples, meaning AI coding assistants could not reliably help with the library — a concrete limitation that slowed development velocity.

**Iteration 4 — Agentic Master Agent (Gemini 2.5 Flash).** The fourth iteration represented the first true agentic attempt: a single master agent given full autonomy to plan and execute all tasks. This introduced genuine decision-making capability but exposed a critical failure mode — the prompts required to express the full HVAC pipeline were too long and complex for the model to follow reliably. A simple while loop was not sufficient to keep the agent on track. This iteration also introduced the CopilotKit frontend, the Agent-to-UI protocol for real-time agent observability, and MCP tools for external tool management.

**Iteration 5 — Plan-Act-Review Architecture (Gemini 2.5 Flash).** Inspired by established multi-agent design patterns, the fifth iteration decomposed execution into three sequential sub-agents: Planner, Actor, and Reviewer. This improved output quality but the agent consistently terminated tasks before completing the full pipeline. Adding a loop agent on top of the PAR structure improved results further but was still insufficient — the agent could not reliably complete all stages in a single run.

**Iteration 6 — Master Agent with PAR Sub-Agent (Gemini 2.5 Flash).** The sixth iteration added a master agent that could choose to act directly or delegate to the PAR structure as a sub-agent. Both the master and the sub-agent were given full autonomy. Despite this added flexibility, the PAR sub-agent still could not produce results of sufficient quality. Frontend tooling was extended with tool call and state variable tracking to provide better visibility into the agent's reasoning.

**Iteration 7 — Master Agent with Hybrid Workflows (Gemini 2.5 Flash).** The seventh iteration replaced the PAR agent with a hybrid approach: specialized image recognition workflows for the drawing analysis stage, combined with an agentic planner for task orchestration. This created an architecture that was neither fully workflow-based nor fully agentic. The recognition quality was not good enough, and the architectural hybrid added complexity without proportional benefit. The entire drawing sequence was ultimately converted back to a workflow,

but results remained insufficient.

**Iteration 8 — Parallel Agents (Gemini 2.5 Flash, then Gemini 3.0 Flash).** The eighth iteration introduced parallel execution: one specialized agent per equipment type running simultaneously, replacing the sequential recognition agents. This significantly improved processing speed. A model upgrade to Gemini 3.0 Flash delivered a meaningful accuracy improvement. Gemini 3.0 Pro was also tested and produced higher quality results but required considerably more time, making it impractical for routine use.

**Iteration 9 — Skills-Based Master Agent (Gemini 3.1 / Claude Sonnet 4.6).** The ninth and final iteration converted all sub-agents into skills — specialized instruction sets executed inline within the master agent's context. This architectural simplification was made possible by the dramatically improved instruction-following capability of the latest models. The agent was also given the ability to visit the frontend via headless browser, capture a screenshot of the live HVAC canvas, and compare it against the source drawing for visual self-correction. The architecture became dramatically simpler, easier to maintain, and produced better results than any previous iteration.

The key insight from this progression is that advances in foundation model capability made simpler architectures viable. The skills-based approach became possible only when models like Gemini 3.1 and Claude Sonnet 4.6 reached sufficient instruction-following capability to execute complex, multi-step domain tasks within a single agent context.

## 4.2 Final Architecture

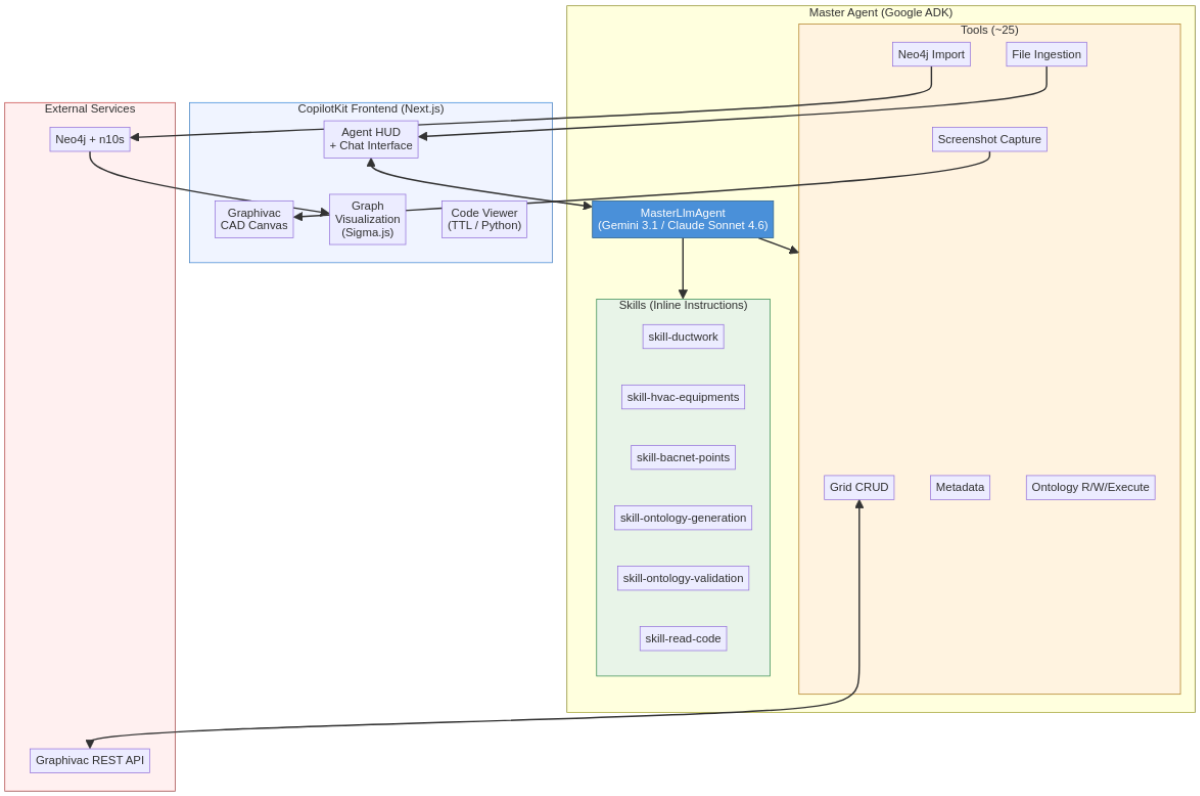


**Figure 3:** SI-Mapper final architecture — skills-based single master agent.

The final architecture organizes SI-Mapper into three main layers that together implement the full pipeline from raw HVAC drawings to a validated ASHRAE 223P semantic model stored in a graph database.

**Frontend (CopilotKit + Next.js).** The frontend serves as a real-time operations dashboard.

A CopilotKit Agent HUD streams the agent's thoughts and actions as they occur, giving operators full visibility into what the agent is doing and why. A Graphivac CAD canvas renders the internal HVAC grid in real time as the agent places ducts and equipment. A Sigma.js graph viewer displays the final semantic model after it is loaded into Neo4j. A code viewer shows snapshots of the generated ASHRAE 223P Python code and the resulting TTL file at each stage of validation.

**Master Agent (Google ADK).** The core of the system is a single *LlmAgent* built on Google's Agent Development Kit, running on Gemini 3.1 or Claude Sonnet 4.6. The agent has access to approximately 25 tools and six skills. It maintains full context across the entire pipeline — from drawing recognition through grid placement, BACnet metadata mapping, and ontology generation and validation — without handing off control to any sub-agent.

**External Services.** Two external services underpin the system. Neo4j with the Neosemantics (n10s) plugin stores the ASHRAE 223P ontology as a native RDF graph and provides Cypher query access for graph traversal. The Graphivac REST API manages the CAD canvas, allowing the agent to read and write the visual grid state bidirectionally.

The choice of a single-agent architecture over the multi-agent sub-agent approaches of earlier iterations was driven by several compounding advantages. A single agent avoids the coordination overhead inherent in multi-agent systems: there is no inter-agent messaging, no shared state synchronization, and no risk of a sub-agent misunderstanding its scope and terminating early. Skills provide domain expertise inline, making the agent's behavior fully observable in a single context window without the opacity of delegated sub-agent calls. The agent naturally maintains full context across all pipeline stages, which is critical for operations such as visual self-correction — where the agent must compare a live screenshot of the canvas against the source drawing and determine what to fix. Finally, the architecture is dramatically simpler to maintain, debug, and extend: adding a new capability means writing a new skill or tool, not redesigning an agent coordination protocol.

### 4.3 Skills

Skills are specialized instruction sets that the master agent executes inline. They are not separate agents — they run inside the master agent's context using the shared tool set. Each skill provides domain-specific knowledge for one stage of the pipeline, encoding expert-level HVAC drawing reading conventions and ontology generation rules that a general purpose language model would not apply consistently on its own.

**Table 5:** Master agent skills and their roles in the pipeline.

| Skill | Purpose |
|---|---|
| skill-ductwork | Identify and place all ductwork from HVAC drawings as the spatial foundation |
| skill-hvac-equipments | Place HVAC equipment (fans, coils, dampers, sensors) onto the established duct s |
| skill-bacnet-points | Extract BACnet point data from CSV exports and map points to grid equipment |
| skill-ontology-generation | Generate ASHRAE 223P Python code from the populated grid state |
| skill-ontology-validation | Iteratively fix and validate ontology code to produce a clean TTL file |
| skill-read-code | Load error-resolution lessons from past coding sessions to avoid repeating mistake |

The skills follow a strict dependency chain that mirrors the physical reality of HVAC commissioning. Ductwork is placed first because all equipment must be anchored to a duct segment — the duct grid is the spatial foundation. Equipment placement follows, using the duct coordinates established in the previous step. BACnet mapping comes next, attaching live point metadata to the equipment already on the grid. Ontology generation reads the fully populated grid and produces Python code that models the system in ASHRAE 223P. Ontology validation then enters an iterative fix loop, executing and correcting the Python code until it produces

a clean TTL file with no runtime errors. Each skill assumes the previous one has completed successfully; the master agent is responsible for detecting failures and deciding whether to retry or report the issue.

### 4.4 Tools

The master agent has access to approximately 25 tools organized into ten categories. Tools handle all interactions with external systems, data storage, and the agent's own operational state — the skills provide the reasoning; the tools provide the execution.

**Table 6:** Master agent tools grouped by category.

| Category | Tools | Description |
|---|---|---|
| Grid Sync | `sync_graphivac_to_agent`, `sync_agent_to_graphivac` | Bidirectional sync between agent state and Graphivac canvas |
| Grid CRUD | `add_component`, `add_components_batch`, `delete_component`, `delete_components_batch`, `read_internal_grid` | Create, read, and delete components in the internal grid |
| Metadata | `write_metadata`, `write_metadata_batch` | Attach BACnet point data to grid equipment |
| Ontology | `search_class_mapping`, `scan_python_files_filtered`, `read_ontology`, `write_ontology`, `execute_ontology`, `read_prompt` | ASHRAE 223P library lookup, code read/write/execute |
| Ontology Signals | `checkpoint_code`, `exit_generator_success`, `exit_generator_failure`, `exit_validator_success`, `exit_validator_failure` | Snapshot versioning and ontology task completion signaling |
| Neo4j | `load_ttl_to_neo4j` | Import validated TTL semantic model into Neo4j graph database |
| Frontend | `capture_frontend_state` | Screenshot the live canvas for visual self-correction |
| Ingestion | `ingest_category_files` | Load user-uploaded files (drawings, CSVs, PDFs) into session |
| State / Progress | `save_agent_state`, `get_agent_state`, `update_step`, `update_status` | Track processing progress and persist state across turns |
| Loop Control | `exit_loop_level_2` | Terminate the master agent session |

### 4.5 Key Technology Decisions

Several technology choices were made deliberately based on lessons from the iterative development process. Each decision reflects a tradeoff between capability, cost, and integration complexity.

**Google ADK.** Google's Agent Development Kit was chosen as the agent runtime for its native Gemini integration and its built-in support for agent lifecycle management, tool registration,

session state, and artifact handling. Earlier iterations used LangChain, which imposed commercial licensing costs for production use; LangGraph was evaluated but abandoned for the same reason. ADK provides the necessary infrastructure at lower total cost and with tighter coupling to the Gemini model family.

**CopilotKit.** CopilotKit implements the Agent-to-UI protocol, enabling the Python ADK backend to stream agent thoughts and actions to the Next.js frontend in real time. Without this library, building the real-time HUD would have required custom WebSocket infrastructure, additional message serialization, and a stateful server — a significant engineering effort with no research value. CopilotKit handles this concern completely, allowing development to focus on the agent pipeline itself.

**Neo4j with Neosemantics (n10s).** ASHRAE 223P is an RDF ontology; its natural storage format is a graph database with native RDF import capabilities. Neo4j with the Neosemantics (n10s) plugin satisfies both requirements: it stores the TTL file as a native graph, and it provides Cypher — a mature, expressive query language — for graph traversal and analysis. Alternative approaches such as storing TTL as flat files or using a triple store were considered, but Neo4j's combination of graph-native storage and strong ecosystem tooling made it the pragmatic choice.

**Sigma.js for Graph Visualization.** Sigma.js renders large graphs using WebGL, providing smooth performance on semantic models with hundreds of nodes and relationships. Heavier alternatives such as D3 or Gephi were considered, but both introduce unnecessary complexity for a browser-native visualization requirement. Sigma.js integrates cleanly with the Next.js frontend and requires no additional server-side rendering accommodations beyond a standard dynamic import guard.

# 5 Results

## 5.1 Experimental Setup

Two HVAC systems were selected for evaluation: System 1 (Air Handling Unit) and System 3 (Air Handling Unit). Both systems are drawn from real buildings with existing BACnet point exports and engineering drawings available for ingestion. The selection ensures that the evaluation reflects production conditions — actual drawings with all the ambiguity and variability of real-world HVAC documentation, not synthetic test cases.

Four metrics are recorded for each AI agent run: total LLM token consumption (sum of input and output tokens across all model calls in the session), estimated token cost in USD at the applicable model pricing, wall-clock time from session start to validated TTL output, and a qualitative assessment of the generated ASHRAE 223P graph based on completeness (all equipment and sensors present), accuracy (correct topology and BACnet references), and interpretability (graph readable by downstream applications).

## 5.2 System 1: Air Handling Unit

**Table 7:** System 1 (AHU) — AI Agent Performance

| Metric | Value |
|---|---|
| Total tokens consumed | **[TODO: Fill after running experiment]** |
| Token cost (USD) | **[TODO: Fill after running experiment]** |
| Wall-clock time | **[TODO: Fill after running experiment]** |
| Model used | **[TODO: Gemini 3.1 / Claude Sonnet 4.6]** |
| ASHRAE 223P graph quality | **[TODO: Describe completeness and accuracy]** |

**[TODO: Insert screenshot of System 1 frontend result]**

### 5.3 System 3: Air Handling Unit

**Table 8:** System 3 (AHU) — AI Agent Performance

| Metric | Value |
|---|---|
| Total tokens consumed | **[TODO: Fill after running experiment]** |
| Token cost (USD) | **[TODO: Fill after running experiment]** |
| Wall-clock time | **[TODO: Fill after running experiment]** |
| Model used | **[TODO: Gemini 3.1 / Claude Sonnet 4.6]** |
| ASHRAE 223P graph quality | **[TODO: Describe completeness and accuracy]** |

**[TODO: Insert screenshot of System 3 frontend result]**

### 5.4 Human Engineer Baseline

For comparison, an experienced HVAC engineer performed the same task manually: reading the same engineering drawings and BACnet point export CSV files, and producing an equivalent ASHRAE 223P semantic model by hand using domain knowledge and available ontology tooling. The human engineer was given the same inputs as the AI agent — no additional context — to ensure a fair comparison of effort and output quality.

**Table 9:** Human Engineer Baseline

| Metric | System 1 | System 3 |
|---|---|---|
| Time to complete | **[TODO: Fill after human test]** | **[TODO: Fill after** |
| Estimated labor cost (USD) | **[TODO: Fill after human test]** | **[TODO: Fill after** |
| ASHRAE 223P graph quality | **[TODO: Describe completeness and accuracy]** | **[TODO: Describe** |

### 5.5 Comparison: AI Agent vs. Human Engineer

**Table 10:** AI Agent vs. Human Engineer Comparison

| | AI Agent | Human Engineer | Speedup |
|---|---|---|---|
| System 1 — Time | **[TODO: AI time]** | **[TODO: Human time]** | **[TODO: Nx]** |
| System 1 — Cost | **[TODO: AI cost]** | **[TODO: Human cost]** | **[TODO: Nx]** |
| System 3 — Time | **[TODO: AI time]** | **[TODO: Human time]** | **[TODO: Nx]** |
| System 3 — Cost | **[TODO: AI cost]** | **[TODO: Human cost]** | **[TODO: Nx]** |

**[TODO: Write analysis comparing AI and human results. Discuss where AI performed well, where it struggled, and implications for production deployment.]**

## 6 Conclusions

### 6.1 Summary of Contributions

This report addressed a documented, economically significant problem in the building industry: manual BACnet point mapping costs 0.5–1.5% of construction value per building, blocks software interoperability, and is identified in peer-reviewed literature and DOE policy reports as a major

impediment to building automation innovation. The same problem directly prevents the large-scale enrollment of commercial buildings into Virtual Power Plant and Non-Wire Alternative programs, because each building requires a custom, per-project integration effort estimated at $5,000–$15,000 before it can participate in grid services. No widely-deployed automated solution for this problem existed prior to this work.

SI-Mapper demonstrates that agentic AI can automate the creation of ASHRAE 223P semantic building models from two raw inputs available on any real building project: HVAC engineering drawings and BACnet point export CSV files. The system combines multimodal drawing recognition — using foundation models capable of interpreting HVAC schematic symbols directly from images — with automated BACnet point-to-equipment mapping and formal ASHRAE 223P ontology generation in a single, end-to-end pipeline. The agent validates its own output through iterative self-correction, looping until the generated Turtle (TTL) file executes without errors and the resulting graph loads cleanly into the Neo4j semantic store. This pipeline replaces a workflow that previously required weeks of expert HVAC engineering labor.

The development process itself is a contribution. Nine experimental iterations, from simple LangChain function-calling in 2024 to the final skills-based architecture on Gemini 3.1 and Claude Sonnet 4.6 in 2026, constitute a systematic record of how agentic AI architectures for domain-specific technical tasks evolved as foundation model capabilities improved. The progression reveals a counterintuitive architectural lesson: simpler architectures became more viable, not less, as models improved. Multi-agent designs with dedicated Planner, Actor, and Reviewer sub-agents — initially necessary to keep the pipeline on track — became unnecessary once models reached sufficient instruction-following capability. The final skills-based single master agent outperformed every multi-agent architecture tried while being dramatically simpler to maintain, debug, and extend.

The technology stack assembled for SI-Mapper — Google ADK, CopilotKit, Neo4j with Neosemantics (n10s), and Sigma.js — provides a reusable reference architecture for agentic AI applications requiring real-time operator observability, semantic graph storage, and interactive graph visualization. The combination of ADK for agent orchestration, CopilotKit for the Agent-to-UI streaming protocol, and Neo4j for native RDF graph storage is not an obvious combination; the design decisions that led to it are documented in Section 4 as a practical guide for future implementors.

### 6.2 Limitations

**Model dependency.** The system's performance is directly coupled to the capabilities and availability of specific foundation models — Gemini 3.1 and Claude Sonnet 4.6 as of this writing. Model API changes, pricing changes, or capability regressions in future model versions could affect pipeline reliability without any change to the SI-Mapper codebase. The skills architecture mitigates this risk somewhat by encapsulating domain knowledge in plain-text instruction sets that can be updated independently of model versions, but the underlying dependency on external model providers remains.

**Limited validation scope.** **[TODO: Update after experiments — describe how many systems were tested (System 1 and System 3), overall accuracy rates, and any cases where the agent failed to produce a valid ASHRAE 223P model on the first or second run.]** The current experimental scope covers a small number of real HVAC systems drawn from a single project context. Statistical confidence in accuracy and generalizability requires a larger validation corpus.

**Domain specificity.** The current skills and tool set are tailored to HVAC air handling units with BACnet points in French-language naming conventions, reflecting the Quebec installation context where this work was developed. Extending SI-Mapper to other building system types (VAV boxes, chiller plants, boiler plants), other control protocols (Modbus, KNX, BACnet/IP vs. BACnet MSTP), or BACnet point naming conventions from other regions and contractors would

require additional skill development and validation. The architecture supports this extension naturally — new skills are plain-text instruction sets — but the domain knowledge required to write accurate skills is non-trivial.

**ASHRAE 223P maturity.** The standard is currently in its second advisory public review and is not yet ratified. The ontology generation logic in SI-Mapper is calibrated against the current advisory draft. Changes to the standard during ratification — particularly to connection point types, medium definitions, or telemetry binding syntax — could require updates to the ontology generation skills and the underlying ASHRAE 223P Python library used to produce the TTL output.

### 6.3 Future Work

**Production validation at scale.** The most important next step is running SI-Mapper against a larger corpus of real buildings — a target of 10–20 HVAC systems spanning multiple system types, contractors, and naming conventions. This would establish statistical confidence in accuracy and cost savings, identify edge cases not encountered in the current development set, and produce the benchmark data needed for peer-reviewed publication.

**Multi-system support.** The current skills cover air handling units. A production-viable system would need skills for VAV terminal boxes, chiller plants, boiler plants, heat pump systems, and mechanical ventilation units. Each system type has its own drawing conventions, point naming patterns, and ASHRAE 223P topology rules. The skills architecture makes this extension tractable: each system type requires a new skill file rather than a new agent or pipeline redesign.

**VPP integration pilot.** The highest-impact near-term application of SI-Mapper output is a demonstration connecting the generated ASHRAE 223P model to a VPP enrollment platform. Such a pilot would demonstrate the complete value chain — from raw HVAC drawings to grid dispatch-ready semantic model — and validate the hypothesis that semantic models are the interoperability layer that eliminates the per-building custom integration step currently blocking VPP scale.

**Automated quality metrics.** The current evaluation of generated ASHRAE 223P models is qualitative. A production evaluation framework would include automated comparison tools: graph diffing against a human-authored ground-truth model, quantitative scores for completeness (fraction of expected equipment nodes present), accuracy (fraction of BACnet references correctly assigned), and topology correctness (fraction of directed connections matching the source drawing). These metrics would make it possible to track regression across model updates and skill revisions without manual inspection.

**Multi-language support.** Adapting the BACnet point mapping skills to recognize English, Spanish, and other naming conventions beyond French would expand SI-Mapper's applicable geography significantly. The mapping skills are the primary location where naming convention knowledge is encoded; adding language variants is a skill-writing task rather than an architectural change.